

Figure 1: When objects have reference

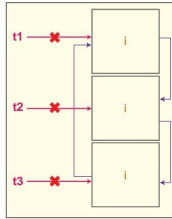


Figure 2: When objects do not have reference

object that has no reference variable pointing to it from outside, but all the objects pointing to each other internally. Let us consider the following code:

```
Class Test {
    Test i;
    public static void main (String[] args){

//Creating three objects*/
    Test t1 = new Test();
    Test t2= new Test();
    Test t3 = new Test();

//Assigning the reference of the all the three objects to
each object's instance variable.*/
    t1.i = t2;
    t2.i = t3;
    t3.i = t1;

//Assigning null to the reference variable*/
    t1 = null;// no object eligible for garbage
collection(GC)
    t2= null;//no object eligible for GC
    t3 = null;// all three objects are now eligible for GC

    }
}
```

In Figure 1 there are three objects, referred to as *t1*, *t2* and *t3*, respectively, and each object has the 'i' instance variable associated with it. These instance variables point to each other's objects and, hence, the bonding between these objects becomes more powerful.

Figure 2 is a bit different. Here, all the three objects are still associated by the internal bonding of the instance variable but, unfortunately, the external bonding is lost by *t1*, *t2* and *t3*; so these three objects become isolated and consequently eligible for garbage collection.

Requesting JVM to run the garbage collector

The method to request JVM to run garbage collection is called the *gc()* method. It is used to give a call to the garbage collector explicitly. The call to the garbage collector by the *gc()* method doesn't mean that the garbage collector is ready to perform garbage collection. The *gc()* method is present inside *java.lang.System* class and *java.lang.Runtime* class.

```
Class Test {
    Public static void main(String[] args){
    Test t = new Test();
    Test t = null;
    System.gc();
    }
    /*Overriding the finalize() method. This method is explained
    in the below section*/
    public void finalize(){
    System.out.println("Garbage collection is performed");
    }
}
```

Finalisation

Finalisation takes place when the *finalize()* method is invoked. The *finalize()* method is called by the garbage collector on an object when garbage collection determines that there are no more references to the object. The method is called by the garbage collector thread each time, before the object is collected as garbage. This is the last option for any object to perform a clean-up task. The *finalize* method is declared in the *java.lang.Object* class. Inside the *finalize* method, you are supposed to write the code which has to be performed before an object is destroyed. It is a best practice to call the *super.finalize()* method once your class frees the resources it was holding. If you don't call *super.finalize()* then any resources held by the super class may never be free. The method is helpful when an object needs to perform some special task before it gets caught by the garbage collector, such as closing the open database connection, open socket or any files. However, if the database connections or files are programmed to be released within a very short period, you will rarely need *finalize*.

```
protected void finalize() {
//-----method code-----
super.finalize();
}
```



By: Vikas Kumar Gautam

The author is a mentor at Emerxve Information Technology (P) Ltd. His main areas of expertise include application development using Java/J2EE and Android for both Web and mobile devices. A Sun Certified Java Professional (SCJP), his interests include acquiring greater expertise in the application space by learning from the latest happenings in the industry. He can be reached at vikash_kumar@emerxve.com